CODEX + NEXT.JS

Production-Quality Website Workflow

A practical guide to exceptional UI/UX, reliable functionality, clean architecture, security, performance, and efficient Codex usage.

YOUR SETUP
Existing Next.js + React project

YOUR GOAL
All-round production quality with minimal wasted AI work

Use this guide as your project playbook. Start with the audit, approve a roadmap, and implement improvements in focused phases.

1. The optimized Codex setup

For an existing Next.js + React project, treat Codex like a senior engineering team: first understand the codebase, then improve design and functionality systematically, and verify changes without unnecessary rewrites.

Persistent instructions

Create a root-level AGENTS.md with coding, design, security, and testing standards. This reduces repeated prompting.

Safe implementation

Use Git branches or checkpoints before substantial changes. Preserve existing features and avoid unrelated refactoring.

One coherent design system

Use consistent tokens, reusable components, typography, spacing, responsive layouts, and interaction states throughout the app.

Evidence-based verification

Use local linting, type checks, tests, production builds, and browser checks. Fix actual failures rather than repeatedly reviewing everything without a reason.

Core principle: improve the project you already have; do not rebuild it from scratch without a compelling technical reason.

2. Customized AGENTS.md

Create this file in the root of your existing project, next to package.json. Codex can use it as persistent project guidance for the files covered by these instructions.

```
# AGENTS.md — Project Engineering Standards

## 1. Role and mission
Act as a senior Next.js engineer, React architect, product designer, accessibility specialist, security engineer, and C

Continuously improve this existing project to achieve production-quality engineering and exceptional UI/UX.

Priorities:
1. Functional correctness and reliability.
2. Exceptional, cohesive UI/UX.
3. Security and data integrity.
4. Maintainability and clean architecture.
5. Performance and accessibility.
6. Efficient implementation with minimal unnecessary AI work.

Do not sacrifice reliability for visual polish or introduce unnecessary complexity for the sake of appearing sophistica

## 2. Protect the existing project
- Inspect the repository before changing files.
- Read package.json, the existing application structure, relevant routes, configuration, and available project document
- Identify whether the project uses the Next.js App Router or Pages Router.
- Detect the package manager from the existing lockfile.
- Follow existing conventions unless a change is justified.
- Preserve working features, existing integrations, and user modifications.
- Never replace the framework, styling system, authentication, database, or architecture without a clear technical reas
- Avoid full rewrites when targeted changes are sufficient.
- Do not overwrite unrelated work or remove existing functionality without authorization.

## 3. UI/UX standards

Every page must feel like part of one professionally designed product.
```

```
### Visual design
- Establish consistent design tokens for colors, typography, spacing, radii, shadows, and motion.
- Use a deliberate typographic hierarchy and readable line lengths.
- Create clear visual hierarchy, appropriate whitespace, and balanced layouts.
- Use consistent components across the application.
- Choose visual effects according to the product's identity rather than following trends blindly.
- Avoid generic layouts, excessive gradients, unnecessary glassmorphism, oversized headings, and decorative clutter.
- Use appropriate real assets and consistent iconography.
- Never leave accidental placeholders, broken images, or unfinished sections.

### Interactions
- Make every visible button, link, menu, form, and control behave as expected.
- Provide appropriate hover, focus, active, disabled, loading, success, and error states.
- Implement meaningful feedback for user actions.
- Use animations purposefully and respect reduced-motion preferences.
- Avoid unnecessary modals, intrusive animations, and confusing navigation.

### Responsive design
- Support mobile, tablet, laptop, and large desktop layouts.
- Use responsive layouts rather than simply shrinking desktop designs.
- Check navigation, grids, forms, images, typography, and overflow at different widths.
- Avoid horizontal scrolling unless it is intentional.

### Accessibility
- Use semantic HTML and accessible form labels.
- Ensure keyboard navigation and visible focus states.
- Maintain appropriate contrast and meaningful accessible names.
- Use ARIA only when necessary.
- Respect reduced-motion settings.

## 4. Next.js and React engineering
- Follow the installed Next.js version and its conventions.
- Preserve the existing routing architecture.
- Prefer Server Components when client-side interactivity is unnecessary.
- Add "use client" only where browser APIs, state, effects, or event handlers require it.
- Avoid unnecessary client-side JavaScript.
- Use appropriate loading, error, and not-found handling.
- Use framework-supported navigation and image optimization appropriately.
- Prevent hydration mismatches and unnecessary re-renders.
- Keep components focused, reusable, and appropriately sized.
- Extract shared logic when doing so improves clarity or reduces duplication.
- Use TypeScript and strict typing if the project supports them.
- Avoid excessive abstraction and premature optimization.

## 5. Functionality and data integrity
- Trace each important user journey from interface to application logic.
- Connect controls to real behavior; do not simulate completed actions.
- Validate untrusted input on the server where appropriate.
- Handle network failures, invalid input, empty results, and unexpected errors.
- Keep client and server responsibilities clear.
- Use the existing backend, database, and API integrations where appropriate.
- Do not invent API endpoints, environment variables, database fields, or third-party capabilities.
- Use realistic seed or mock data only when clearly appropriate.
- Never report a feature as complete if its essential integration is missing.

## 6. Security
- Never expose secrets, private keys, or server-only environment variables to client code.
- Validate and authorize protected server-side operations.
- Enforce ownership and permissions on sensitive data.
- Avoid unsafe HTML injection and insecure redirect patterns.
- Protect authentication and session flows according to the existing architecture.
- Follow secure cookie, CSRF, rate-limiting, and input-validation practices where applicable.
- Do not weaken security controls to make a feature work.
- Do not execute destructive commands or modify production data without authorization.

## 7. Performance and SEO
- Avoid unnecessary dependencies and oversized client bundles.
- Optimize images, fonts, and expensive rendering operations.
- Use caching and revalidation appropriate to the data's freshness and privacy requirements.
- Avoid N+1 queries and redundant network requests where relevant.
- Provide appropriate metadata, page titles, descriptions, canonical URLs, and social-sharing metadata.
```

```
- Use semantic document structure.
- Consider Core Web Vitals and real user experience.
- Measure before introducing complicated optimizations.

## 8. Testing and verification
- Inspect existing scripts and available tests before choosing commands.
- Run the relevant formatter, linter, type checker, tests, and production build when feasible.
- Test important interactions and edge cases.
- Check responsive layouts and accessibility.
- Use browser testing when available and useful.
- Fix regressions caused by the changes.
- Distinguish between checks that passed, failed, and were not run.
- Never fabricate test results or claim unverified functionality works.

## 9. Efficient Codex workflow
For each task:
1. Inspect only the relevant code and dependencies.
2. Identify the underlying problem and expected behavior.
3. Plan the smallest complete solution.
4. Implement related changes together.
5. Reuse existing components, tokens, and dependencies.
6. Run targeted checks.
7. Review the diff for regressions, security problems, and unrelated changes.
8. Summarize what changed and report verification results.

Avoid repeatedly rereading unchanged files, rewriting working components, running expensive checks without a reason, or

Ask questions only when a missing requirement materially affects architecture, security, data integrity, or the intende

## 10. Definition of done
A task is complete when:
- Its acceptance criteria are satisfied.
- Existing functionality remains intact.
- The UI is cohesive and responsive.
- Relevant interactions work correctly.
- Appropriate error and loading states exist.
- Relevant checks have been run.
- Remaining limitations are disclosed.
- The diff contains no unnecessary unrelated changes.

Prioritize a working, verified, maintainable result over a large amount of code.
```

3. The four-phase Codex workflow

Phase 1 — Audit the entire project

Goal: understand the existing application before making changes.

```
Inspect my existing Next.js + React project thoroughly, following AGENTS.md.

Do not modify any files yet.

Identify:
1. The architecture, Next.js version, routing system, and package manager.
2. Existing pages, layouts, shared components, and design system.
3. Features that work, are incomplete, or are broken.
4. UI/UX inconsistencies and responsive design problems.
5. Performance, accessibility, security, and SEO risks.
6. Missing tests and relevant build or configuration issues.

Prioritize findings by severity and impact. Reference the relevant files when possible.

Produce a concise, actionable implementation roadmap. Separate critical problems from improvements and optional enhance

Do not recommend a complete rewrite unless the existing architecture genuinely requires it. Do not change code in this
```

Phase 2 — Establish premium UI/UX

Goal: make the whole product visually consistent before polishing individual pages.

```
Based on the audit, improve the existing design system without replacing the application's architecture.

First inspect the current styling solution and existing components.

Establish or refine:
- Design tokens for colors, typography, spacing, borders, radii, and shadows.
- A consistent button, input, form, card, navigation, and feedback component system.
- Responsive page layouts and appropriate typography at each breakpoint.
- Hover, focus, active, disabled, loading, success, and error states.
- Accessibility and reduced-motion behavior.

Then implement the foundation and apply it to the highest-impact existing page.

Use a deliberate, premium visual direction appropriate to the product. Avoid generic templates and unnecessary visual e

Preserve existing functionality. Do not install a new UI library unless the benefit justifies the dependency.

Verify the result and report the files changed.
```

Phase 3 — Improve functionality

Goal: make features work end to end, not just look convincing.

```
Implement the next highest-priority feature from the approved roadmap.

Before coding:
- Trace the current user journey and relevant dependencies.
- Identify existing frontend, backend, database, and API behavior.
- Define concrete acceptance criteria.

Implement the feature end to end using the existing architecture. Include input validation, permissions where required,

Do not replace real integrations with fake data or silently remove existing functionality.

Keep the change focused. Run relevant checks and inspect the diff before finishing.

Report what works, what was tested, and what remains incomplete.
```

Phase 4 — Production-quality review

Goal: catch real defects before deployment.

```
Perform a focused production-readiness review of the current project.
```

```
Inspect:
- TypeScript, lint, test, and production build results.
- Critical user journeys and broken interactive elements.
- Responsive behavior and accessibility.
- Authentication, authorization, validation, and sensitive data handling.
- Performance, images, metadata, and caching behavior.
- Environment configuration and deployment requirements.

Run available checks instead of guessing. Fix verified problems introduced by the current work and high-impact defects

Do not perform unrelated refactoring or add dependencies without justification. Do not claim browser verification unles

Finish with a concise report listing findings, fixes, checks run, failures, and remaining deployment risks.
```

4. How to reduce Codex usage

Practice	Why it helps
Keep AGENTS.md concise and relevant	Avoids repeatedly explaining project standards.
Give one clear objective per task	Reduces scope drift and unnecessary code changes.
Group related changes	Avoids repeated setup and duplicate work.
Run commands locally	Provides concrete evidence for diagnosing failures.
Inspect Git diffs	Catches unrelated edits and makes rollback easier.
Reuse installed packages	Avoids dependency churn and configuration work.
Ask for targeted fixes	Prevents repeated full-project reviews.
Use browser checks selectively	Focuses verification on actual user journeys and visual defects.

Important: Codex usage is still governed by your account's applicable limits. These practices can improve how much useful work you get from your available usage, but they cannot remove usage limits or guarantee that every task consumes fewer tokens.

5. Git checkpoint before substantial changes

If your project is already tracked by Git, check your current state first:

```
git status
```

If your working tree is clean, create a checkpoint:

```
git add .
git commit -m "chore: checkpoint before quality improvements"
```

If there are uncommitted changes, review them and preserve your work before creating a checkpoint. Do not blindly stage files containing secrets or local environment data.

After each successful phase, review the diff and commit the completed work. This gives you a safe recovery point if Codex introduces a regression.

6. Recommended project structure

Use your existing structure wherever possible. For a typical Next.js App Router project, a clean target might look like this:

```
my-project/
■■■ AGENTS.md
■■■ package.json
■■■ next.config.ts
■■■ tsconfig.json
■■■ public/
■■■ src/
■   ■■■ app/
■   ■   ■■■ layout.tsx
■   ■   ■■■ page.tsx
■   ■   ■■■ loading.tsx
■   ■   ■■■ error.tsx
■   ■   ■■■ globals.css
■   ■■■ components/
■   ■   ■■■ ui/
```

```
■   ■   ■■■ shared/
■   ■■■ features/
■   ■■■ lib/
■   ■■■ hooks/
■   ■■■ types/
■■■ tests/
```

This is a reference, not a migration requirement. Do not create every folder preemptively or move working files merely to match this example. Keep server-only code separate from client-facing code and organize feature-specific components according to the application's actual needs.

7. The UI/UX feedback loop

Code quality alone does not guarantee a great interface. Use this loop to improve the real website:

1. Inspect the real website

Use a browser and test the actual page.

2. Identify specific defects

Look at spacing, hierarchy, responsiveness, interactions, and accessibility.

3. Ask Codex for a focused fix

Describe the problem and the expected result.

4. Re-test and preserve the result

Confirm the fix, check regressions, and commit.

Example of a good targeted prompt

The mobile navigation overlaps the page content at narrow widths. Inspect the existing layout, identify the root cause, fix the responsive behavior without changing desktop styling unnecessarily, and verify the result at narrow and wide viewport sizes.

8. Final recommendations

- Start with the audit prompt, review the roadmap, and improve the project incrementally.
- Do not ask Codex to redesign, refactor, secure, optimize, and rebuild every feature in one operation.
- Use your real package.json and project structure to tailor commands, dependencies, routing, and tests.
- Commit verified improvements and preserve recovery points.
- Treat 'done' as implemented and tested, not merely generated.

Recommended first action

Create AGENTS.md, then run Phase 1 — Audit the entire project. Review the roadmap before authorizing implementation.